ReactiveDynamics.jl Modeling Copilot

A guided workflow that takes a capacity-constrained business decision from a conversation to a checked, reproducible model and a number you can defend — without anyone on the team writing a line of code.

The analyst describes the decision. The copilot builds the model, tries to break it, and reports a number that says what it rests on, and the seed that replays it.

Decisions of this kind — a launch-capacity reservation, a bid under a staffing ceiling, a portfolio that competes for the same people — today need a modeler, a quarter, and trust in a spreadsheet nobody else can read. The copilot turns that into a guided session an analyst or a strategy lead can run, with every step checked by the engine rather than by the reader's faith.

what it is a reusable modeling capability: an interview, a model of your constrained operation, a health pass that can refuse, and a briefing — run in conversation

who runs it an analyst, a strategy lead or a technical executive; no Julia, no scripting, no simulation background

why it can be trusted the model is a checked data document, not generated code — validated before it runs, reproducible from a seed afterwards

the case on this page a launch-supply decision in pharma, blind-captured and actually simulated; the company is invented, the numbers are measurements from paired single-seed paths, not ensembles

---

LINES OF CODE YOU WRITE

0

the model is a structured document authored in conversation and checked before it runs

GATED PHASES

4

frame, build, try to break it, report — each ends in a file you can read, and the third may stop the rest

PLAUSIBILITY CHECKS

9

they ask whether what you called scarce actually binds; a failure blocks the headline number

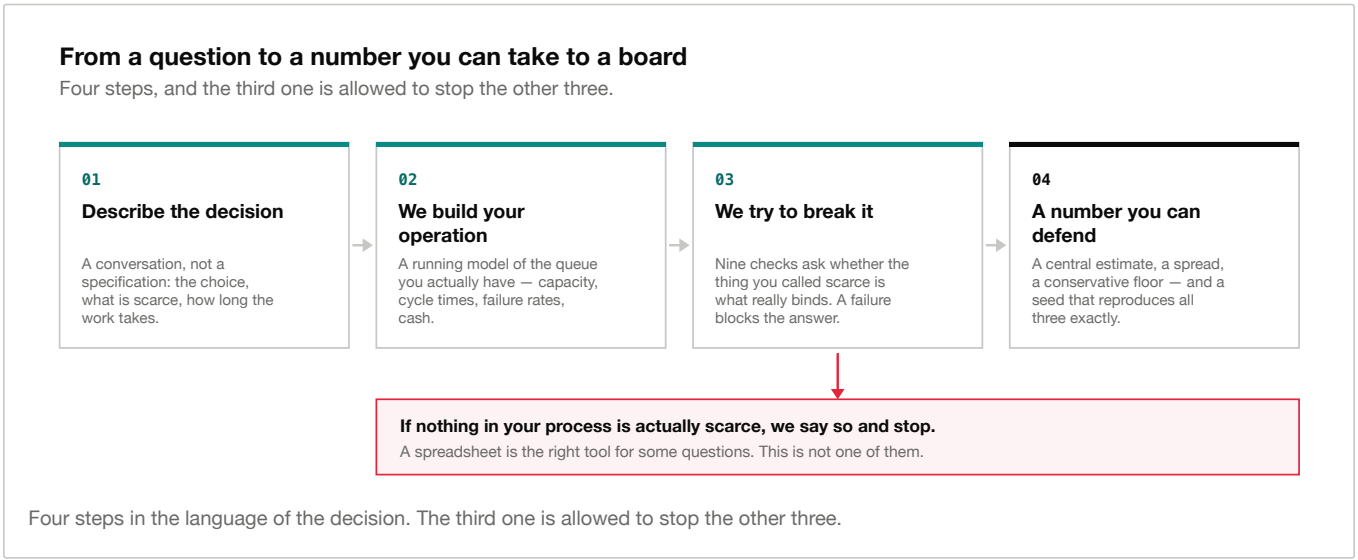
SEED REPLAYS IT

1

same model, same seed, same answer — bit for bit, months later, on another machine

The expensive part of a capacity decision is not the arithmetic. It is getting the question right, getting the operation right, and knowing which parts of the answer you are entitled to defend in the room. Those steps are why these questions go to a specialist, take weeks, and come back as a spreadsheet the decision-maker has to take on faith.

The copilot does that work with you. It interviews you about the decision, turns your description of the operation into a running model of it, stress-tests that model against what you said the business does, and only then reports a number, together with the mechanism that produced it and an honest account of how much evidence stands behind it. The same capability applies to the next question, and the one after that: this is a way of working, not a one-off model.



Four steps in the language of the decision. The third one is allowed to stop the other three.

1 · THE VALUE: DECISIONS AT SCALE, WITHOUT A MODELING BOTTLENECK

More decisions get a real model. Today a queueing question — who waits when the line is full, what a reservation is worth before the readout, what twelve more people buy — gets a model only when it is big enough to justify a specialist. Most get a spreadsheet, and a spreadsheet cannot represent a queue: contention is an interaction between rows, and its rows are independent by construction. The copilot lowers the cost of a real model to a conversation, so the threshold drops.

The person who owns the decision stays in the loop. The interview is conducted with the analyst or the executive, in business language. Every assumption lands in a written brief they agreed to — including which inputs were their own estimates and what they expect the answer to be — so the result is compared against the expectation rather than quietly replacing it.

The answer is defensible, not just plausible. Each number arrives with the evidence behind it (a spread and a conservative floor when the options were run as ensembles, the named example paths when they were not), the seed that reproduces it, and the command that produced it. When the CFO asks "what if the second line costs more", the answer is one edited input and a re-run, not a rebuilt workbook.

It compounds. A study is a directory — the brief, the model, the health report, the briefing, every command. The next question about the same operation starts from that model rather than from a blank sheet, and the method is the same whoever runs it.

2 · HOW IT WORKS, IN FOUR STEPS

Step	What happens	What you get	Gate
Frame	a structured interview — the decision, the lever, what is scarce, how long the work takes, what counts as an answer	a written brief, agreed before anything is built	something has to actually bind
Author	your operation becomes a running model: the pools work waits for, the steps that hold them for a while and sometimes fail, the cash — one version per option, each an *arm*	a checked model document per arm	the validator returns no errors
Diagnose	nine checks ask whether the model does what you said the business does, and whether the scarce thing really binds	a health report, every failure stated	a red check blocks the number
Report	every option is run on the same random outcomes, as an ensemble or on named example paths; differences are read against their own noise	a briefing: estimate, mechanism, and what it rests on	every number carries its evidence and a seed

It opens with an interview, not a template. The first step asks what a good modeler would ask: which decision is actually on the table, what the alternative really is (not a straw man), what is scarce, what happens to work that cannot start yet, how long each step takes and how often it fails. Numbers the user is unsure of are written down as estimates, and the published audit of each captured session says which of them were tested and which were not.

The comparison is designed before it is run. What a decision needs is the difference between two futures. Each future becomes an arm — the same model with one thing changed — so the answer is measured as a difference against its noise, not argued between two separately built spreadsheets.

The model has to survive being doubted. If nothing in the operation is actually contended, there is no queue to simulate, and the copilot says so and stops.

3 · WHY THE NUMBER CAN BE TRUSTED

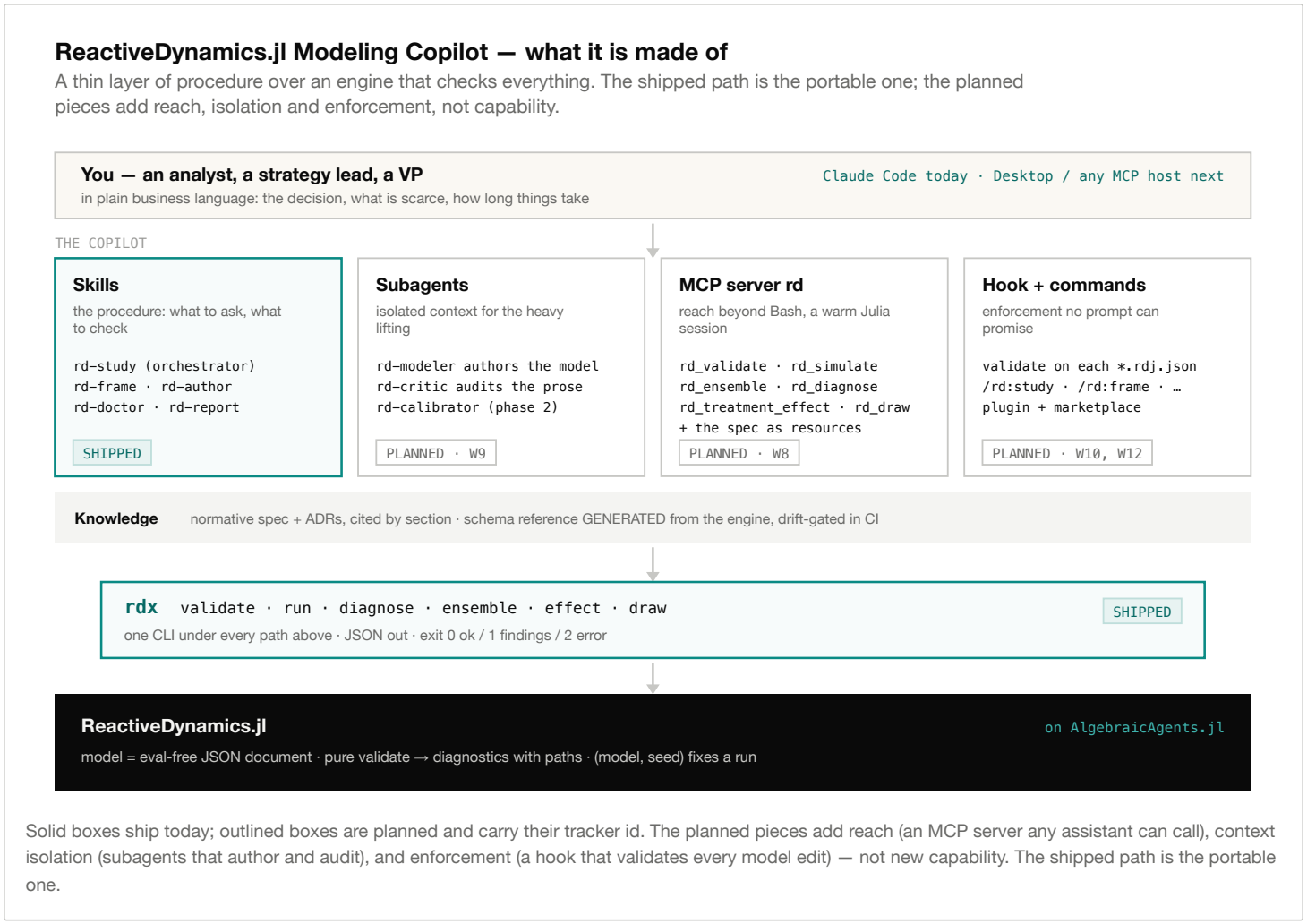
The model is data, not code. The operation ends up as a structured document in a closed vocabulary the engine understands — places that hold resources, transitions that consume them over time, rates, costs, what each arm changes. Nothing in it is a program. That single design choice is what makes the rest possible:

- **Checked before it runs.** A pure validator reads the document and returns a list of problems, each with the exact path of what is wrong. A malformed model does not produce a plausible wrong number; it produces diagnostics. That is what makes it safe to put a language model in the authoring seat: it proposes, a deterministic checker disposes, and the loop repeats until the checker is silent.
- **Nothing is executed that was generated.** No `eval`, no generated scripts, no shell in the model. If a study needs custom logic it is one declared file, announced in the run log.
- **Checked again, for meaning.** A schema cannot tell you that your bottleneck never binds or that the horizon cuts off the payoff. Nine plausibility checks do, and a failure blocks the headline.
- **Reproducible by construction.** A run is fixed by the model and a seed. Options are compared on the same random outcomes, so a difference is the option, not the luck.
- **Honest about uncertainty.** Every published figure says what it rests on: an ensemble with a spread and a floor, or a stated handful of paths. A difference that cannot be told from noise is reported as a tie rather than rounded into a story.
- **Guarded in CI.** The reference the copilot reads is generated from the engine itself and fails the build if it drifts, and a known-answer test re-derives a published result through the same path on every change.

Contrast with a code-writing assistant. An assistant using a general simulation library writes code, and code cannot be checked before it runs: it crashes or it returns *something*, and *something* is the dangerous case. Here the machine can author the model, be wrong, be told exactly where, and try again — and a reviewer who reads no code at all can still audit the result.

4 · WHAT IT IS MADE OF

The copilot is a thin layer of procedure over an open-source engine that checks everything. **Skills** carry the procedure for each phase — what to ask, what to check, how to report. A single command-line tool, `rdx`, does the work: validate, run, diagnose, run ensembles, measure the effect of an option, draw the network. Underneath is ReactiveDynamics.jl, a stochastic, resource-constrained process engine built on AlgebraicAgents.jl.



Solid boxes ship today; outlined boxes are planned and carry their tracker id. The planned pieces add reach (an MCP server any assistant can call), context isolation (subagents that author and audit), and enforcement (a hook that validates every model edit) — not new capability. The shipped path is the portable one.

How you use it today. Open a session in Claude Code on a checkout of the repository and say what you are deciding. The `rd-study` skill takes over: it runs the interview, writes the brief, authors the model, runs the checks, and asks for your sign-off at each gate. You never type a command; you see each one, and its result, in the conversation.

How you will use it next. The MCP server (W8) exposes the same six operations as tools with a warm Julia session, so any MCP host — Claude Desktop included — can drive a study without a terminal. The plugin (W12) makes installation one step. None of this changes what a study is; it changes who can start one.

5 · A REAL SESSION, FROM QUESTION TO ANSWER

The question. A VP of launch readiness has two Phase-3 biologics reading out nine months apart, one sterile fill-finish line that can carry one launch but maybe not two, a CMO term sheet that expires on 30 November 2026, and a CFO who says not to pay for capacity on a 45% asset. Reserve a CMO slot now, build a second line, or wait? The operator arrived leaning toward a reservation *sized for Tern*.

What the copilot did. In three rounds of questions it recovered the constraint the operator said “people forget”: a fill site must run three validation batches before the filing, or wait six months for a supplement. It then built one model with an arm per option, checked it against the operator's own calibration figures ($250.0M of Kestrel revenue in its first year against their ~$250M), and compared the options on paired paths, where the same seed gives every option the same readout dates and lead times.

What it found.

- **Reserve the slot now, usable by either product, and review it at Kestrel's readout: +$3.8M ± $1.1M against waiting** at the stated odds, and +$25M if a batch earns its peak-year value.
- **The value is supply protection.** If both succeed (27%), the reservation is worth about +$39M. Waiting leaves Tern short at launch on 26 of 37 paths, costing on average 11 batches and $51M. Otherwise the reservation is a bounded $9–11M cost.
- **The operator's prior was overturned.** A Tern-only reservation leaves Kestrel short on every path where both succeed.
- **Their other prior was confirmed.** A second line is $73–105M worse than waiting even when both succeed.

The moment worth watching. The first pass came out at roughly break-even, because a rigid reservation carried a $72M tail when only Tern succeeds. The model made that tail visible, and the operator supplied the fix: Kestrel reads out nine months earlier, so drop the reservation if Kestrel fails. The copilot encoded the rule, re-ran 30 paired paths and reported the result, including where the operator's guess was right.

How to read the evidence. Every comparison is a set of paired single-seed paths, weighted by the stated odds. No option-level ensembles were run, because a 96-seed ensemble took 53 minutes (tracked as W24). The session was blind-captured by two agents that could not see each other's reasoning, and the operator's pre-commitment fact sheet is published with an audit. That audit leads with what the interview left untested: demand ±30%, marketed growth and readout correlation.

The excerpt below is the actual exchange, with its tool calls, model documents and the delivered chart. The full transcript is `agent/sessions/launch_supply.html`.

PHASE 1 — FRAME

OPERATOR

I need a decision on fill-finish capacity for executive committee in about five weeks, and I want it to hold up when our CFO pushes back. Two Phase-3 biologics are coming. Kestrel reads out around March next year, and Tern reads out around the end of 2027. We have one sterile fill-finish line in-house, and most of it is already taken by our marketed products. It can probably carry one launch but not two. There are three options on the table:

1. Pay a CMO a reservation fee now to hold a fill-finish slot.
2. Build a second line of our own.
3. Wait for the readouts and decide then.

My problem is lead time: tech transfer, PPQ and the build all take longer than the gap between readout and launch. Our CFO says we shouldn't pay for capacity on an asset that has less than even odds. What I need from you is a recommendation I can defend, what it costs, what it protects us against, and what would change the answer. I don't care about the modeling details. I care about whether we launch with product on the shelf.

CLAUDE

Phase 1 of 4: framing the decision. I'll need about three rounds of questions from you, then I build the model, check that it behaves sensibly, and bring you a recommendation.

Is simulation worth it here? Yes. All three tests pass:

- **Capacity is tight.** One sterile line, mostly taken by marketed products. It can carry one launch, not two.
- **Things take time.** Tech transfer, PPQ and a new build all take longer than the gap between readout and launch. That is exactly why "wait and decide" is risky.
- **A choice gets forced along the way.** If both assets succeed, and their timing overlaps, someone has to decide who gets the in-house line and who goes short.

Your CFO's argument ("don't pay for capacity on an asset with less than even odds") treats each asset on its own. The real risk sits in the scenario where *both* succeed and collide on one line. A per-asset spreadsheet can't price that. A simulation that plays out the readouts, the lead times and the line allocation thousands of times can. It will also tell you if the CFO is right, and if so I'll say so.

First round. Rough numbers are fine, and a range is better than a guess:

1. **Odds and value.** What probability of success do you put on Kestrel and on Tern, and which one is under 50%? What is a month of lost or delayed launch worth for each: first-year sales, peak sales, or a per-month figure? And what units do you use (for example $M, months)?
2. **Lead times from the day you commit.** How long until usable capacity for (a) a CMO slot, meaning reservation, tech transfer and PPQ, and (b) a second in-house line, meaning build, qualification and PPQ? And how long from readout to planned launch for each asset? My working assumption: a CMO is faster but not fast enough, and a build is slower still. Correct me.
3. **The cheques.** What are the CMO reservation fee, the build cost, and any running costs? Do you lose the fee if the asset fails, or can you cancel or reassign the slot to the other asset? That last point matters a lot for your CFO's

argument.

After that I'll have a smaller second round: how the in-house line gets shared if both succeed, and what result would change your mind.

▸ RUN 3 commands

```
$ read agent/skills/rd-study/SKILL.md
  → orchestrator: 4 gated phases (frame/author/diagnose/report), the qualifying test, the rdx CLI contract
$ read agent/skills/rd-frame/SKILL.md
  → phase-1 slot table + pre-registration table
$ read agent/studies/portfolio_bid/BRIEF.md
  → exemplar brief, for shape only
```

3 EXCHANGES OMITTED Two rounds of questions: odds, readout timing, the validation-batch rule, line capacity and rejects, batch demand per product, the CMO terms and the build estimate. The assistant's arithmetic on the operator's own numbers found that the sales and batch forecasts disagree by about 2× per batch, and asked which to trust.

CLAUDE

Phase 1 of 4: framing, one step from done. I've written up the brief. Before you sign it off, there's one thing you need to see, because it changes the question.

Your own numbers say the in-house line carries Kestrel alone for only about its first year. This is arithmetic on your figures, taking 3% off for rejects, not model output yet:

When	Spare good batches/month after marketed	Kestrel needs	Tern needs
Kestrel launch, Jul 2028	2.6	1.0	–
Tern launch, Apr 2029	2.3	2.5	1.0
Kestrel +12 months, Jul 2029	2.1	3.0	1.4
Kestrel peak, Jul 2031	1.1	4.5	3.4

So the question isn't really "one launch or two". It's where Kestrel's growth, and all of Tern, gets made. The CMO's 4 batches a month roughly matches either Tern's peak or Kestrel's growth, but not both. The late-horizon gap is also where your S&OP growth estimate matters most. If this table is wrong, for example because marketed volume is moving elsewhere or a debottleneck is planned, tell me now.

The setup I'll build:

What	How it's represented
In-house line	12.8 batches/month; 3% rejected; marketed products always served first
Readouts	Kestrel 60% (plus a 50% re-test), Mar 2027 slipping 0–2 months; Tern 45%, Dec 2027 ±3 months; independent
Regulatory clock	File 3 months after readout, 12 months review, launch 1 month later. A site not in the BLA needs a 6-month PAS and ships only after it's approved

Tern's filing site	Without the CMO: 3 PPQ batches on our line, competing with Kestrel's stock build. With the CMO: the CMO, and filing waits if its PPQ runs late
Launch stock	6 months of forward demand (Kestrel ~12 batches, Tern ~10), built in the 6 months before launch
Who waits	Marketed, then Kestrel, then Tern
Shortfall	Unmet demand is lost. If Kestrel is short in its first 6 months, it also loses 15% of peak share for good
CMO terms	$5M/yr in advance, $0.4M/batch, 4 batches/month, ready 12–15 months after signing. Walking away after a failure costs two annual fees ($10M, the top of your range)
Options compared with waiting	(1) CMO now, reassignable to Kestrel; (1b) CMO now, Tern only, to cover the unconfirmed term sheet; (2a) build now; (2b) build after readouts if both succeed; (3) wait, then take the CMO's mid-2028 opening if both succeed
Answers	Expected NPV vs waiting at 9% through 2032, with its range; and the odds of a short Kestrel or Tern launch under each option

Every option sees the same readout outcomes and dates in each simulated scenario, so the differences come from the decision and not from luck.

Three open items. My defaults are in brackets:

1. **Second-line capacity.** How many batches a month would the new line do? [Default: the same as today's, 12.8.] And how is the $120M phased? [Default: spread evenly over the build.]
2. **Your sales and batch forecasts disagree by about 2x.** In year 1, $250M over ~23 Kestrel batches is about $11M per batch. At peak, $1.2B over 54 batches is about $22M. Tern: ~$7M vs ~$19M. Either the early batches include channel or sample stock, or one forecast is off. [Default: value each shipped batch at the peak rate ($22M Kestrel, $19M Tern, before the 85% margin). That makes a lost early batch look more expensive, and I'd re-test at the year-1 rate.] Which do you trust?
3. **Lost or delayed?** When a patient can't be supplied in a given month, do we lose that month's sales for good, or do they wait? [Default: lost, with a delayed-sales variant only if you think it matters.]

Once you answer these or accept the defaults, I start building (phase 2).

▸ RUN 3 commands

```
$ read agent/reference/model_schema.md · agent/skills/rd-author/SKILL.md · spec/CONTRACT_DRAFT.md
  → unmet spawn proposals are dropped, not queued — so a shortfall must be modelled as lost demand, explicitly
```

```
$ python3 -c '<good-batch headroom (-3% rejects) vs Kestrel/Tern batch demand, by month>'  exit 0 · ok
  → Jul-2028 K launch   headroom 2.61  K need 1.00  T need 0.00  gap(K+T) +1.61
  Apr-2029 T launch   headroom 2.25  K need 2.50  T need 1.00  gap(K+T) -1.25
  Jul-2029 K+12       headroom 2.12  K need 3.00  T need 1.38  gap(K+T) -2.25
  Jul-2031 K+36 peak  headroom 1.07  K need 4.50  T need 3.44  gap(K+T) -6.87
```

```
$ python3 -c '<implied value per batch: sales forecast ÷ batch forecast>'  exit 0 · ok
  → Kestrel yr1 10.9 $M/batch; peak 22.2 $M/batch
  Tern    yr1  7.4 $M/batch; peak 18.8 $M/batch
```

`agent/studies/launch_supply/BRIEF.md (draft for sign-off; three OPEN items)`

OPERATOR

Your table is the thing I was worried about, and it's worse than I'd said out loud. As far as I know there's no debottleneck planned and no marketed volume moving elsewhere. So yes, build it on that table. Given that, change option 3: if Kestrel alone succeeds, "wait" should also be able to take the mid-2028 CMO opening. By your numbers Kestrel outgrows the line by mid-2029 even without Tern. Otherwise you're comparing against a straw man, and the CFO will spot it.

1. **Second line.** Use 12 batches a month effective, which is the engineering pre-study figure, not 12.8. For phasing, a build like this is usually back-loaded, roughly 20/50/30 over three years. That's my estimate, and evenly spread is fine if it doesn't move the answer.

1. **Which forecast.** Use the year-1 rate as the base case, not the peak rate. I'm not certain which forecast is off. My estimate is that the early batches carry free-goods and bridge-program supply and heavier launch discounts, so a year-1 batch really is worth less. The bigger reason: if the case for the CMO rests on valuing a lost launch batch at $22M, the CFO will say we inflated the benefit. Show the peak rate as the upside sensitivity. If the recommendation only holds at the peak rate, I need to know that.

1. **Lost or delayed.** Lost. In immunology an unsupplied patient goes to a competitor or stays on their current therapy. For Tern, those patients aren't going to wait for us either. You don't need the delayed variant.

On the $10M walk-away cost: using the top of the range is fine, it's the conservative choice. Also show the $5M case, because that's the number I'd quote in the room.

With those changes, go ahead and build it.

PHASE 2 — AUTHOR · PHASE 3 — DIAGNOSE

CLAUDE

AUTHOR (phase 2 of 4) is done. The model is built, checked and matched to your numbers.

In your "both succeed" world with no reservation, we launch Kestrel with product on the shelf and **Tern is the one that launches short.** Tern ends up on the losing side of your priority rule. One sample path shows how:

- Tern's qualification batches wait behind Kestrel's launch-stock build on our line, so Tern files 2 months late.
- In Tern's first 12 months it gets about 5 batches against the roughly 20 it needs.
- Kestrel loses nothing.

That is one path, not a forecast; the probabilities come from the full run. The same pattern shows up everywhere I tested. Kestrel only ever ran short in one setup (a Tern-only CMO slot with both assets succeeding), and even there not in its first six months. So the 15% share-loss penalty may almost never trigger. If that holds, the case for the CMO is about protecting Tern, not Kestrel.

What is in the model, in your terms:

- **Wait** now takes the mid-2028 CMO opening if either readout succeeds, as you asked.
- **Reserve now** makes the CMO Tern's filing site from day one. Kestrel can use the slot too (via the prior-approval supplement) if Tern fails. If both assets fail, we walk away for $10M, with $5M as a sensitivity.
- **Tern-only reservation**: if Tern fails, it now falls back to the mid-2028 opening for Kestrel. Without that it would have been a straw man.
- **Second line** at 12 batches/month, capex phased 20/50/30 (even spread as a check). Two versions: approved now, or after Tern's readout if both succeed.
- Base case values batches at the year-1 rate, with the peak rate as the upside test. Lost sales are gone, not delayed. Kestrel is also re-tested at 50% odds.
- **Calibration**: Kestrel's first 12 months come out at $250.0M and 54 batches a year by the end. Tern's first 12 months come out at $151.1M. You gave $250M, 54 and $150M.

I also fixed two bugs found while checking: the first-year CMO fee and the first month of capex were not being charged.

Two questions before I run the full set of scenarios:

1. **Who waits when the line is full?** Your rule is marketed, then Kestrel, then Tern. Taken literally, Kestrel keeps a full six months of stock on the shelf before Tern gets a single batch.

- The alternative: Kestrel's launch stock and this month's demand come first, then Tern, then Kestrel's buffer top-up.
- On the same sample path, the alternative gives Tern about 14 batches in its first year instead of 5, and Kestrel still loses nothing.
- *Proposed default:* your literal rule as the base case, with the alternative shown next to it, because ExCo may ask why Kestrel's buffer beats Tern's launch.

1. **Could we hold two CMO contracts?** If we reserve the slot for Tern only and both readouts succeed, could Kestrel also take the mid-2028 opening as a second contract?

- *Proposed default:* no. One reservation, and the mid-2028 opening only serves as the fallback if the first contract ends.

With those two answers I'll run all options with the same readout outcomes in every option, so the differences between options are like-for-like. Then comes phase 3, the numbers with error bars.

Everything is in `/Users/bima/ReactiveDynamics-review/agent/studies/launch_supply/` : `BRIEF.md` (updated with your turn-4 decisions), `DOCTOR.md` (the checks), `capture_log.md` (turn 4 logged) and `gen_models.py`, which writes the 11 scenario files. All of it is committed.

▸ `MODEL` **the base model — wait for the readouts** `48 places · 31 transitions · 78 arcs · 27 rules`

`agent/studies/launch_supply/model.rdj.json` — the complete document, as it ran

▸ `MODEL` **arm: reserve the CMO slot now** `48 places · 31 transitions · 78 arcs · 27 rules`

the whole difference from `model.rdj.json`

```
--- model.rdj.json
+++ arm_cmo.rdj.json
@@ -88,7 +88,7 @@
   },
   {
    "name": "cmo_now",
-   "value": 0.0
+   "value": 1.0
   },
   {
    "name": "cmo_reassign",
```

▸ **MODEL** **arm: build a second line now** 48 places · 31 transitions · 78 arcs · 27 rules

the whole difference from `model.rdj.json`

```
--- model.rdj.json
+++ arm_build_now.rdj.json
@@ -100,7 +100,7 @@
   },
   {
    "name": "build_now",
-   "value": 0.0
+   "value": 1.0
   },
   {
    "name": "build_after",
```

▸ **RUN** 5 commands

```
$ python3 agent/studies/launch_supply/gen_models.py   exit 0 · ok
  → wrote 11 documents (one network; arms differ only in params)

$ for f in agent/studies/launch_supply/*.rdj.json; do rdx validate $f; done   exit 0 · ok
  → all 11: error_count 0

$ rdx run arm_cmo__both.rdj.json --seed=1 --trace=k_sold,t_sold,k_age,t_age,own_used,cmo_used   exit 0 · ok
  → K first 12 shipping months: 230 tenths, revenue $250.0M   (user: ~$250M)
  K last-12-month sold: 540 tenths = 54 batches/yr  (user: 54)
  T first 12 months: revenue $151.1M   (user: ~$150M)

$ rdx run model__both.rdj.json --seed=1 --trace=…,own_used,cmo_used   exit 0 · ok
  → own line tenths/month: 92 94 94 … 127 128 128 127 … 128   (128 available: the line binds)
  Tern first 12 months 49 tenths; Tern lost 935 tenths; t_file_delay 2; Kestrel lost 0

$ rdx diagnose <each arm and forced-outcome copy> --seed=1 --expect-binds=own_free --expect-
reusable=line,cmo,line2 --tally=…   exit 1 · findings
  → model | 7 pass · 1 warn · 1 fail | nonpass: transitions_fire (design-dead branches, see DOCTOR.md),
  horizon_sufficient
  arm_build_now | 6 pass · 2 warn · 1 fail | + bounded_pools: cmo_free unused when a second line exists
```

`agent/studies/launch_supply/gen_models.py` `agent/studies/launch_supply/DOCTOR.md` `agent/studies/launch_supply/BRIEF.md` `(turn-4 decisions)`

1 EXCHANGE OMITTED The operator corrects the allocation rule (patients first, not the literal marketed → Kestrel → Tern order) and adds that the reserved slot can carry Kestrel if Tern fails.

PHASE 4 — REPORT

CLAUDE

Phase 4 of 4 (Report). My recommendation is to sign the CMO reservation before 30 November, with the slot usable for either asset. Don't build a second line, now or after the readouts. Only tie the slot to Tern alone if you can get the second CMO contract in writing.

How the numbers were made. Ensembles were cancelled, so none of these figures is an ensemble average. Each option was simulated on a few example paths in each of the four readout outcomes (both succeed, Kestrel only, Tern only, neither). There are 3 paths per option when both succeed and 1 path in each other outcome, with the same paths for every option. The four outcomes were then weighted by your odds (Kestrel 0.60, Tern 0.45). I measured no spread, so no figure has an error bar. Where the paths disagree, I say so.

Money: the CFO is right. Reserving now is worth +$0.9M against waiting ($847.9M vs $847.0M), which is a tie. Taking the three both-succeed paths one at a time, the gain is +$5.7M, –$12.2M or +$9.3M, so even its sign is not settled.

What the fee buys: Tern's launch.

- The chance of a supply-short Tern launch drops from 0.18 to 0.00.
- The chance of Tern filing late because its PPQ is stuck behind Kestrel drops from 0.18 (2 months late) to 0.00.
- If both readouts fail, the reservation costs $9.4M.

The decision for ExCo is whether to pay up to about $9M of downside to take Tern's shortage risk from about 1 in 5 to zero.

Option	eNPV vs wait ($M)	Chance Tern short at launch	Chance Tern files late	If both fail ($M)
Wait	—	0.18	0.18	0.0
Reserve now, both assets	+0.9	0.00	0.09 *	–9.4
Reserve now for Tern, second slot later	+0.3	0.00	0.09 *	–9.4
Build after the readouts	–23.0	0.18	0.18	0.0
Build now	–105.8	0.18	0.18	–123.9

* This remaining late filing (3 months) is not caused by Kestrel. On one path Tern read out early and the CMO was not yet qualified.

Your turn-5 questions:

- **Kestrel at launch:** it is never short in its first six months, under any option or sensitivity.
- **Is Kestrel ever at risk?** Only in one setup, which I've flagged: slot reserved for Tern and no second contract. There Kestrel runs short on every both-succeed path. It loses about 102–107 batches of demand, all after its first six

months. The weighted chance is 0.27.

- **Does "reserve for Tern now, add a second slot later" beat "reserve for both"? No.** With the second contract it ties (+$0.3M vs +$0.9M). Without it, it is the worst CMO option: −$72.5M, while reserve-for-both is +$16.3M.
- **Check before signing:** can Kestrel move into the reserved slot if Tern fails? That is your reading of the term sheet, not confirmed.

What would change the answer (reserve-now for both, against waiting):

- **Stronger:** valuing batches at the peak rate gives +$20.4M. With no second contract it is +$16.3M, and +$61.1M at peak value. Your literal allocation rule gives +$8.8M.
- **Weaker:** at Kestrel PoS 0.50 it ties at −$1.5M, and Tern's protection is unchanged.
- **Small effects:** a $5M walk-away instead of $10M adds about +$1.0M. Spreading build capex evenly costs $0.95M more. The 2-month Kestrel floor made no difference.
- **Building** comes out negative in every case I ran.

Caveats:

- Each outcome other than "both succeed" rests on one path.
- Everything stops at end-2032, so the build options are understated by whatever the line earns after that.
- The completed 96-seed baseline ensemble was sent as a file I'm not allowed to read, so I did not use it. Paste its result into the conversation if you want it as a cross-check.

Possible next steps: get the second contract's terms in writing (this decides between the two reservation structures); rerun the single-asset outcomes on two more seeds each (12 runs, about 8 minutes); test whether letting Tern's PPQ fall back to our own line removes the last 3-month delay.

▸ RUN 5 commands

```
$ python3 agent/studies/launch_supply/gen_models.py  exit 0 · ok
  → wrote 22 documents + 44 forced-outcome scenario copies

$ run_scenarios.sh <arm> <world> <seed> …   # rdx run scenarios/<arm>__<world>.rdj.json --seed=S --
trace=cash,k_sold,t_sold,k_lost,t_lost,…  exit 0 · ok
  → 51 run files, every one ok: true, findings []
  both s1..s3 × 11 arms · konly s1 × 8 · tonly s1 × 5 · neither s1 × 5

$ python3 agent/studies/launch_supply/analyze.py   # worlds weighted by the odds: both .27, konly .33, tonly
.18, neither .22  exit 0 · ok
  → model              eNPV 847.0   vs wait   +0.0 | P(T short launch) 0.18
  arm_cmo            eNPV 847.9   vs wait   +0.9 | P(T short launch) 0.00
  arm_cmo_tern_only  eNPV 847.2   vs wait   +0.3
  arm_build_after    eNPV 824.0   vs wait  −23.0
  arm_build_now      eNPV 741.1   vs wait −105.8

$ python3 agent/studies/launch_supply/paired.py  exit 0 · ok
  → arm_cmo vs model, both-world d s1..s3  +51.9  −14.3  +65.2 | weighted per path +5.7 −12.2 +9.3

$ rdx draw scenarios/model__both.rdj.json --exec-map --seed=1  exit 0 · ok
  → ok: true, layer exec_map
```

agent/studies/launch_supply/figures/exec_map_wait_both_s1.svg (execution map, wait option, both succeed, seed 1)

Editorial note — the run budget changed mid-session. Phase 4 was first planned as 96-seed ensembles per option. The first one took 53 minutes on this machine, because `rdx ensemble` rebuilds and recompiles the model for every seed (tracked as W24). The operator cancelled the batch after the baseline finished and set the budget to single-seed runs only. So every comparison below is a set of **paired example paths** — the same seed gives every option the same readout dates and lead times — with readout outcomes forced and weighted by the stated odds, not an ensemble. The one finished 96-seed ensemble (wait only) was later passed to the assistant as a cross-check.

2 EXCHANGES OMITTED

The operator challenges the first report on three points. 90 more runs take the both-succeed case to 37 paired paths. The assistant retracts two of its own claims out loud, a “0.18” late-filing figure and a “worst case $9–10M” framing, after finding a $72M loss on one Tern-only path.

THE OPERATOR CHANGES THE RULE

OPERATOR

This is what I can use. The corrections on "on time" and the worst case are exactly what I'd have been caught on. I won't quote late-filing numbers at all until you've resolved the mismatch with the baseline.

But your Tern-only result doesn't match how I'd actually run the business. Kestrel reads out in March 2027, nine months before Tern. If Kestrel fails, our line suddenly has plenty of room. I wouldn't keep paying $0.4M a batch at the CMO and bet Tern's filing on their qualification schedule. I'd move Tern's PPQ in-house, file Tern on our own line, and give the CMO notice. That means eating the $5–10M. So the reservation isn't a fixed bet on Tern; it's something we decide to keep or drop at Kestrel's readout. Model it that way:

- **Kestrel fails:** Tern's PPQ goes in-house, we give CMO notice, and we pay the forfeit.
- **Kestrel succeeds:** Tern stays at the CMO, as you have it now.

My guess is that this cuts the Tern-only loss down to roughly the both-fail cost, and takes the $72M tail away. If I'm wrong, show me.

On your questions:

1. **Backup qualification.** Regulatory will tell me it's feasible to list both sites in the BLA if both have PPQ data at filing. My estimate is about $0.5M a batch all-in including drug substance, so around $1.5M for three batches. The real cost, though, is three batches of line time right in the middle of Kestrel's stock build, which is the same collision you already found. So the backup only makes sense when Kestrel has failed. That's the rule above. Don't model the backup as a separate option in the both-succeed world unless it's cheap to do.

1. **Run budget.** Spend it on the Tern-only outcome, with the exit-at-Kestrel-readout rule built in. The no-second-contract sensitivity can wait, because I'm taking the "Tern-only" structure off the table unless we have the second contract in writing.

If the rule change brings the Tern-only loss into line, I'll take this version to ExCo: reserve now, review at Kestrel's readout. Send me that result and the final one-paragraph wording, and we're done.

CLAUDE

Phase 4 of 4, the final result. Your guess holds: under the review rule, the Tern-only outcome costs $9.4M against waiting, and the $72M tail is gone.

The rule: reserve the CMO now and review at Kestrel's readout. If Kestrel fails, Tern's validation batches move in-house, Tern files on our own line, we give the CMO notice and pay the forfeit. If Kestrel succeeds, nothing changes.

- **Tern-only outcome, 30 paired paths:** –$9.4M ± $0.02M against waiting. Every path falls between –$9.7M and –$9.3M.
- It is pure cost. Tern's revenue is the same as under waiting on all 30 paths, Tern files on time on all 30, and no Tern demand is lost on any of them.

- The old rule (keep the CMO no matter what) lost $31.7M ± $6.0M in this outcome over 10 paths. Four of those 10 lost more than $20M, the worst lost $72.1M, and 4 of 10 filed late.
- **Every outcome except both-succeed now costs about the same:**

Outcome	Cost vs waiting	Paths
Kestrel only	$8.8M	3
Tern only	$9.4M	30
Both fail	$9.4M	3

The 3-path outcomes vary by less than $0.1M across their paths.

- **Expected value:** +$3.8M ± $1.1M against waiting (plausible range +$1.6M to +$6.0M). This now passes the brief's test that the value minus two standard errors stays above zero. Last turn it failed, at –$1.1M ± $3.4M.
- At the peak value per batch: +$25.0M ± $2.8M.
- If Kestrel's odds of success are 0.50: +$1.6M ± $0.9M, which just misses the test (–$0.2M at the low end).
- **What the review rule does not change:** when both succeed, the rule never fires. The value there stays at +$38.7M ± $4.1M over 37 paths, and Tern's launch is fully supplied on all 37 (under waiting, Tern is short on 26 of 37). The remaining downside is also in that outcome: 3 of 37 paths lose money, the worst $14.3M. That happens when Tern reads out before the CMO has qualified and has to wait for it.

The late-filing mismatch is resolved. It was sampling chance, not a model error.

- I re-ran three members of the 96-seed baseline with their exact seeds. They reproduce its late-filing months and lost Tern demand exactly, with or without the outcomes forced.
- The value gap is a constant 0.72% from a one-month discounting convention, which cancels out of every comparison.
- Our 13 late filings in 37 paths against the baseline's 2 in 20 fits chance ($p = 0.06$). Pooled, the rate is 15 in 57 (range 0.17–0.39). The two sources do differ on how long a late filing runs: 2–3 months in our paths, 2 months in both of the baseline's cases.
- Late filing is about as likely either way: 0.10 overall under reserve-and-review, 0.095 under waiting.
- The figures are usable now, but quote them with their range: they are the least precise numbers in the report.

Backup qualification: I did not model it at your $0.5M a batch. The model charges $0.15M a batch for in-house validation, so its cost in the Tern-only outcome is understated by about $1M. That is arithmetic, not a simulated result.

One paragraph for ExCo:

> We recommend reserving the CMO fill-finish slot now, before 30 November 2026, usable by either Kestrel or Tern, and reviewing it at Kestrel's readout in March 2027. If Kestrel fails, Tern's validation batches move to our own line, Tern files there, and we give the CMO notice and pay the forfeit. If Kestrel succeeds, the reservation stands. Modelled over paired simulated paths, this is worth about +$3.8M (±$1.1M) against waiting at the stated odds, and about +$25M if a batch earns its peak-year value. Its purpose is supply protection. If both assets succeed (27% chance), it is worth about +$39M and removes a Tern launch shortfall that otherwise occurs on 26 of 37 paths, costing on average about 11 batches and $51M. If either asset fails, the cost is a bounded $9–10M: over 30 paths in the Tern-only case, and 3 each for Kestrel-only and both-fail. We do not recommend

> building a second line: it is about $84–87M worse than waiting even when both succeed (8 paths). We do not recommend a Tern-only reservation unless the second CMO contract is in writing. The one remaining risk is a late CMO qualification delaying Tern's filing if both succeed: 3 of 37 paths lose money, the worst about $14M.

▸ **MODEL** **arm: reserve now, review at Kestrel's readout (the operator's rule)** 48 places · 31 transitions · 78 arcs · 27 rules

the whole difference from `model.rdj.json`

```
--- model.rdj.json
+++ arm_cmo_exit.rdj.json
@@ -88,7 +88,7 @@
   },
   {
    "name": "cmo_now",
-   "value": 0.0
+   "value": 1.0
   },
   {
    "name": "cmo_reassign",
@@ -128,7 +128,7 @@
   },
   {
    "name": "exit_at_k",
-   "value": 0.0
+   "value": 1.0
   },
   {
    "name": "cmo_exited",
```

▸ **RUN** 4 commands

```
$ python3 agent/studies/launch_supply/gen_models.py   # + arm_cmo_exit: at Kestrel readout, if Kestrel failed, move Tern PPQ in-house, give notice, pay forfeit  exit 0 · ok
  → wrote 23 documents + 48 scenario copies
  rdx validate arm_cmo_exit.rdj.json: ok, 0 findings
  regeneration check: stored runs arm_cmo both s1, model tonly s2 re-run byte-identical

$ run_scenarios.sh arm_cmo_exit both,konly,neither 1 · arm_cmo_exit tonly 1..30 · model tonly 4..30 · arm_cmo tonly 4..10 · 6 ensemble replays (members 20, 96, 6; forced and unforced)  exit 0 · ok
  → 75 runs this turn, every one exit 0

$ python3 agent/studies/launch_supply/exit_review.py  exit 0 · ok
  → inertness check, arm_cmo_exit vs arm_cmo on the same seed (must be 0): both 0.0 · konly 0.0 · neither 0.0
  TERN-ONLY WORLD, paired against wait
   arm_cmo_exit vs wait: mean -9.40 se 0.02 n 30; min -9.66 max -9.25 | Tern files late 0/30; Tern demand lost on 0/30
   arm_cmo      vs wait: mean -31.67 se 5.95 n 10; min -72.09 max -18.32 | Tern files late 4/10
  WEIGHTED 'reserve now, review at Kestrel readout' vs wait
    both +38.71 (n 37) | konly -8.76 (n 3) | tonly -9.40 (n 30) | neither -9.38 (n 3)
    weighted delta +3.81, se 1.10; -2se +1.62, +2se +6.00

$ python3 agent/studies/launch_supply/ens_check.py   # replay 3 members of the 96-seed wait ensemble on their exact seeds  exit 0 · ok
  → member 20: forced == unforced True; delay 2 t_lost 92 (ensemble 2 / 92); eNPV ratio 1.00721 (1.09^(1/12) = 1.00721)
  member 96: forced == unforced True; delay 2 t_lost 59 (ensemble 2 / 59); ratio 1.00724
```

```
  member 6:  forced == unforced True; delay 0 t_lost 91 (ensemble 0 / 91); ratio 1.00723
  late filing, wait, both succeed: ours 13/37, ensemble 2/20; Fisher exact two-sided p = 0.059
```

`agent/studies/launch_supply/exit_review.py` `agent/studies/launch_supply/ens_check.py` `agent/studies/launch_supply/FINDINGS.md` `(revision 3)`

OPERATOR

Good, that's the version I'll take to ExCo. Three edits to the paragraph before I use it:

1. **Kestrel's odds.** Add a sentence saying that at 50% it's roughly break-even, around +$1.6M. That was the probability I said I actually believe. I'd rather say it myself than have the CFO find it.
2. **Build numbers.** Your "about $84–87M worse" doesn't match the –$73M to –$105M range across the 8 paths you gave me last turn. Quote the range, not what looks like an average.
3. **The failure cost.** Say "$9–11M", to cover the roughly $1M of in-house validation cost you said is understated.

Everything else I'll say as written: reserve before 30 November, usable by either product, review at Kestrel's readout, don't build, no Tern-only reservation without the second contract in writing, and the one risk left is the CMO qualifying late. On my side, legal confirms that the slot can be reassigned, and regulatory confirms that Tern can be filed on our line if Kestrel fails. If either of those comes back no, I'll come back to you.

Thanks. This will stand up in the room.

CLAUDE

I checked all three edits against the runs, and all three are in the paragraph below. The build range needed one fix.

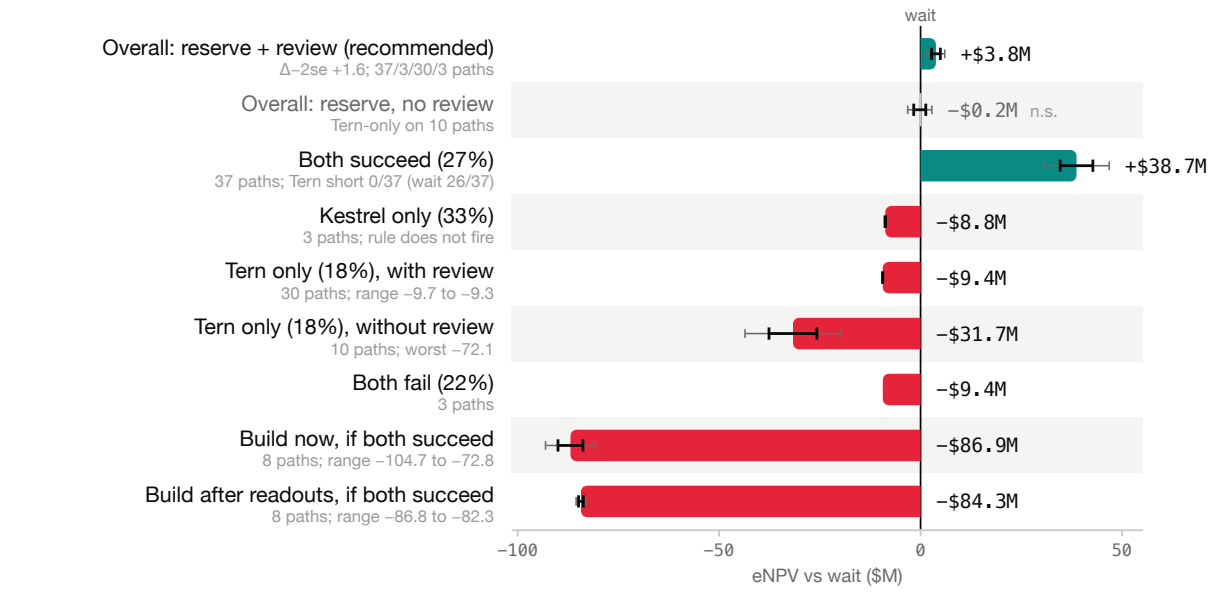
1. **Kestrel at 50%:** correct. At 50% the reservation comes to +$1.6M ± $0.9M against waiting. The low end of the range is slightly below zero (–$0.2M), so "roughly break-even" is fair.
2. **Build numbers:** your –$73M to –$105M is the range for building now only. Building after the readouts has its own, narrower range: –$82M to –$87M. Both are over 8 paths where both products succeed, and the paragraph now gives both ranges.
3. **$9–11M:** safe to say. The measured failure costs are $8.8–9.7M. I also need to correct something I told you last turn. The model charges our in-house validation the same way whether we wait or reserve, because in both cases our line is Tern's only filed site. A higher in-house cost would hit both options, so it mostly cancels in the comparison. That means the understatement is much less than the $1M I quoted. The $9–11M still covers the measured losses, with room to spare.

Final paragraph for ExCo:

> We recommend reserving the CMO fill-finish slot now, before 30 November 2026, usable by either Kestrel or Tern, and reviewing it at Kestrel's readout in March 2027. If Kestrel fails, Tern's validation batches move to our own line, Tern files there, and we give the CMO notice and pay the forfeit. If Kestrel succeeds, the reservation stands. Modelled over paired simulated paths, this is worth about +$3.8M (±$1.1M) against waiting at the stated odds, and about +$25M if a batch earns its peak-year value. At a 50% chance of Kestrel succeeding it is

> roughly break-even, around +$1.6M. Its purpose is supply protection. If both products succeed (27% chance at the stated odds), it is worth about +$39M and prevents a Tern launch shortfall that otherwise occurs on 26 of 37 paths, costing on average about 11 batches and $51M. If either product fails, the cost is a bounded $9–11M. We do not recommend building a second line. Even when both products succeed, it is $73–105M worse than waiting if built now and $82–87M worse if built after the readouts (8 paths each). We do not recommend a Tern-only reservation unless the second CMO contract is in writing. The one remaining risk is the CMO qualifying late: if both succeed, Tern's filing can be delayed, and 3 of 37 paths lose money, the worst about $14M.

This rests on two answers you are still waiting for. The model assumes legal is right that the slot can be reassigned, and that regulatory will allow Tern to be filed on our own line if Kestrel fails. If either comes back no, this recommendation would need to be re-run. Thank you, and good luck in the room.



The delivered chart: each option against waiting, paired single paths, odds-weighted overall and by outcome.

▸ RUN 3 commands

```
$ python3 agent/studies/launch_supply/exit_review.py --pk=0.5   exit 0 · ok
  → weighted delta +1.61, se 0.91; −2se −0.22, +2se +3.44

$ python3 agent/studies/launch_supply/paired_both.py | grep build   exit 0 · ok
  → arm_build_now:   both-world d vs wait mean −86.9 n 8 range −104.7..−72.8
  arm_build_after: both-world d vs wait mean −84.3 n 8 range −86.8..−82.3

$ julia agent/scripts/chartkit.jl agent/studies/launch_supply/figures/options_vs_wait.json   exit 0 · ok
  → chartkit: wrote figures/options_vs_wait.svg — bars_error, self-contained (from the turn-8 numbers; no new
  runs)
```

`agent/studies/launch_supply/FINDINGS.md (final)`

FROM ONE STUDY TO A CAPABILITY

What is shipped: the engine, the four-phase workflow as five skills, the `rdx` CLI with JSON output and exit codes, the generated schema reference with its CI drift gate, the known-answer acceptance test, and four published studies, three of them with their captured sessions.

Next	What it adds	Tracker
MCP server `rd`	the six operations as tools, a warm session, the spec as resources — any MCP host, no terminal	W8
Subagents	`rd-modeler` authors in an isolated context; `rd-critic` audits the briefing's prose against the numbers	W9
Validate hook	every edit to a model document is validated automatically — enforcement a prompt cannot promise	W10
Study card	a one-page, shareable summary of a finished study	W11
Plugin	one-step install, slash commands, marketplace distribution	W12

These are sequenced to follow endorsement: the capability is demonstrated on the shipped path, and the planned pieces widen who can reach it.

WHAT WE ARE NOT CLAIMING

- **The case on this page is composed, not a client result.** The company and its products are invented. Every number is a real measurement from a real model, and the command that produced it is published alongside it.
- **A model is only as good as the facts put in.** The interview asks for the ones that move the answer, writes down which ones were estimates, and reports what changes if an estimate is wrong.
- **This does not replace a valuation or an S&OP process.** It answers the question they cannot: what the queue does to the value.
- **It does not help with every decision.** If nothing is scarce and nothing takes time, a spreadsheet is the better tool, and the copilot says so.
- **The planned components are planned.** The MCP server, subagents, hook and plugin are drawn on this page so the design is visible; they are not claimed as built.

The companion pages are `agent/overview.html` (the component division, the integration surface, the full `portfolio_bid` session) and `agent/copilot_deck.html` (this argument as slides). Every number in the case comes from `agent/studies/launch_supply/`, where each is listed next to the command that produced it. The engine is ReactiveDynamics.jl.